Twin layer operations console guide

This guide documents the operations console for Layer 1 (the Digital Twin) in the TSYP 14 City Brain Convergence architecture. It explains the user interface layout, the live incident timeline, telemetry visualizations, audit capabilities, and the red team adversary controls.

---

1. Interface tour and visual design

The operations console runs with dark theme styling (`#0b0f14` background, `#121821` card surfaces, `#e6edf3` body text, `#3b82f6` primary accents). Telemetry and state indicators adhere to functional semantic coloring:
- Nominal: `#22c55e` (green)
- Contained or isolated: `#f59e0b` (amber)
- Attacked or diverging: `#ef4444` (red)
- Actuator refused or blocked: `#a855f7` (purple)

Top status bar and operational KPIs

The main visible heading reads **Twin operations console**. Directly beneath it, the top status bar provides a single-line overview of the runtime environment:
- Sim time: Full RFC 3339 timestamp formatted in monospace without truncation (`YYYY-MM-DDTHH:MM:SSZ`).
- Speed: Configured simulation multiplier relative to real time (for example, `20x`).
- Throughput: Current wire publication velocity in messages per second (nominal ~420 msg/s).
- API: Online or offline indicator for the Twin Red Team REST API (`Twin online` or `Twin API down`).
- Kafka: Online or degraded indicator for the telemetry streaming broker (`Kafka ok` or `Kafka {state}`).
- MCP: Online or offline indicator for the synchronous tool-call server (`MCP up` or `MCP down`).
- Recent callers: Identities observed calling MCP tools within the last 5 simulation minutes (such as `brain` , `guardian` , or `scenario`).

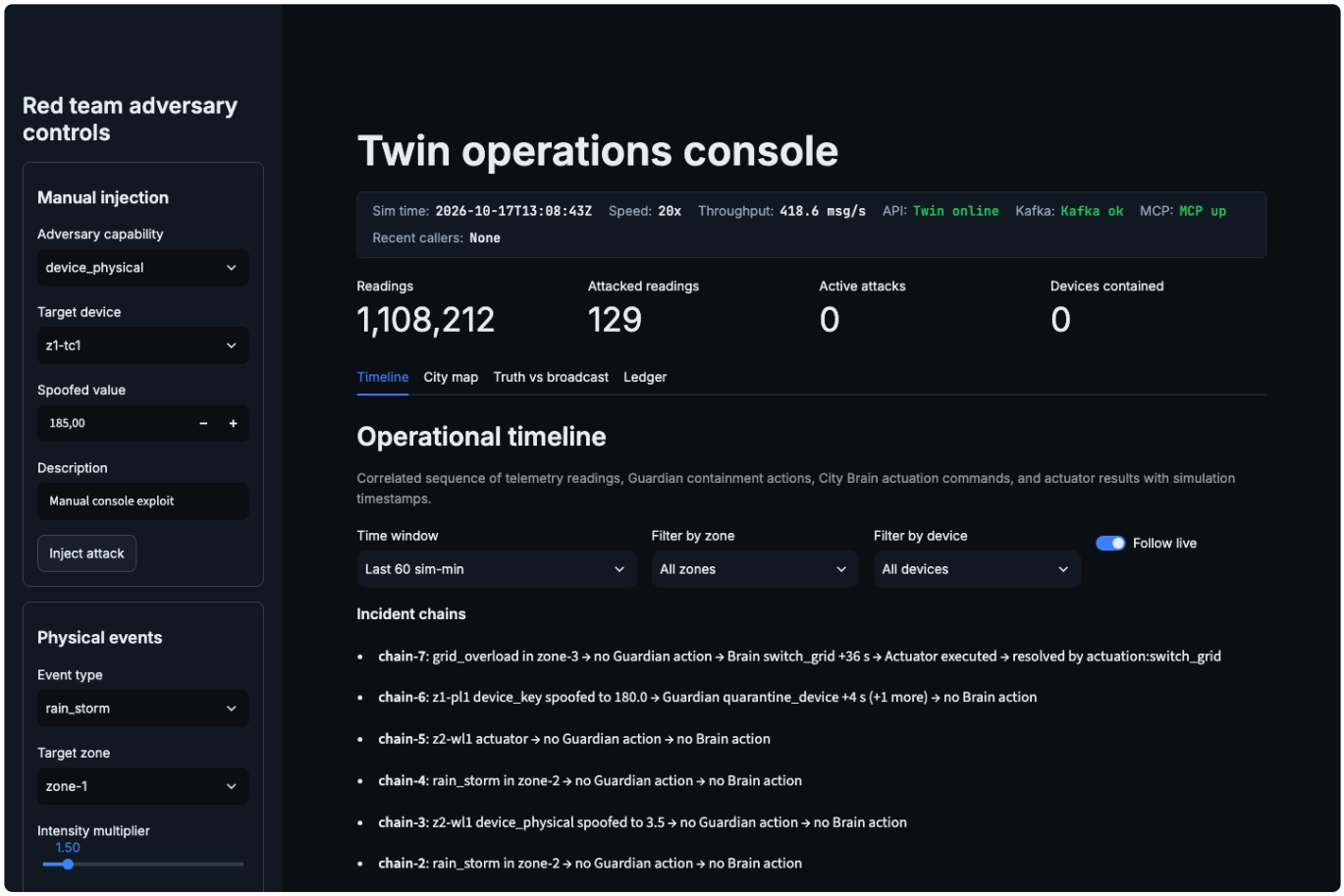
Below the status bar, four headline KPI tiles display key operational metrics:
- Readings: Total count of logged telemetry records.
- Attacked readings: Total volume of records marked under active attack.
- Active attacks: Number of active adversary capabilities currently executing.
- Devices contained: Total number of devices under quarantine or isolation.

---

2. Console tabs

The console organizes monitoring and audit functions across four tabs.

Tab 1: Timeline



Operations console incident timeline

The timeline tab is the primary operational view. It correlates events across five distinct lanes:
1. Twin lane: Physical event lifecycles (start, escalation, resolution), adversary attack injections and clearances, and the first attacked reading for each incident.
2. Guardian lane: Automated containment and countermeasure actions, including quarantine, isolation, release, reinstatement, and reading rollback operations.
3. Brain lane: Supervisory command decisions, including incident dispatch and valve adjustments.
4. Actuator lane: Physical actuation execution results, distinguishing successful operations from refused commands (such as refusal under an actuator tampering attack).
5. Test harness lane: Diagnostic actions originating from scenario test runners (such as device reinstatement or settle commands), kept collapsed in an expander by default to preserve operational clarity.

Incident chains and operational data grid

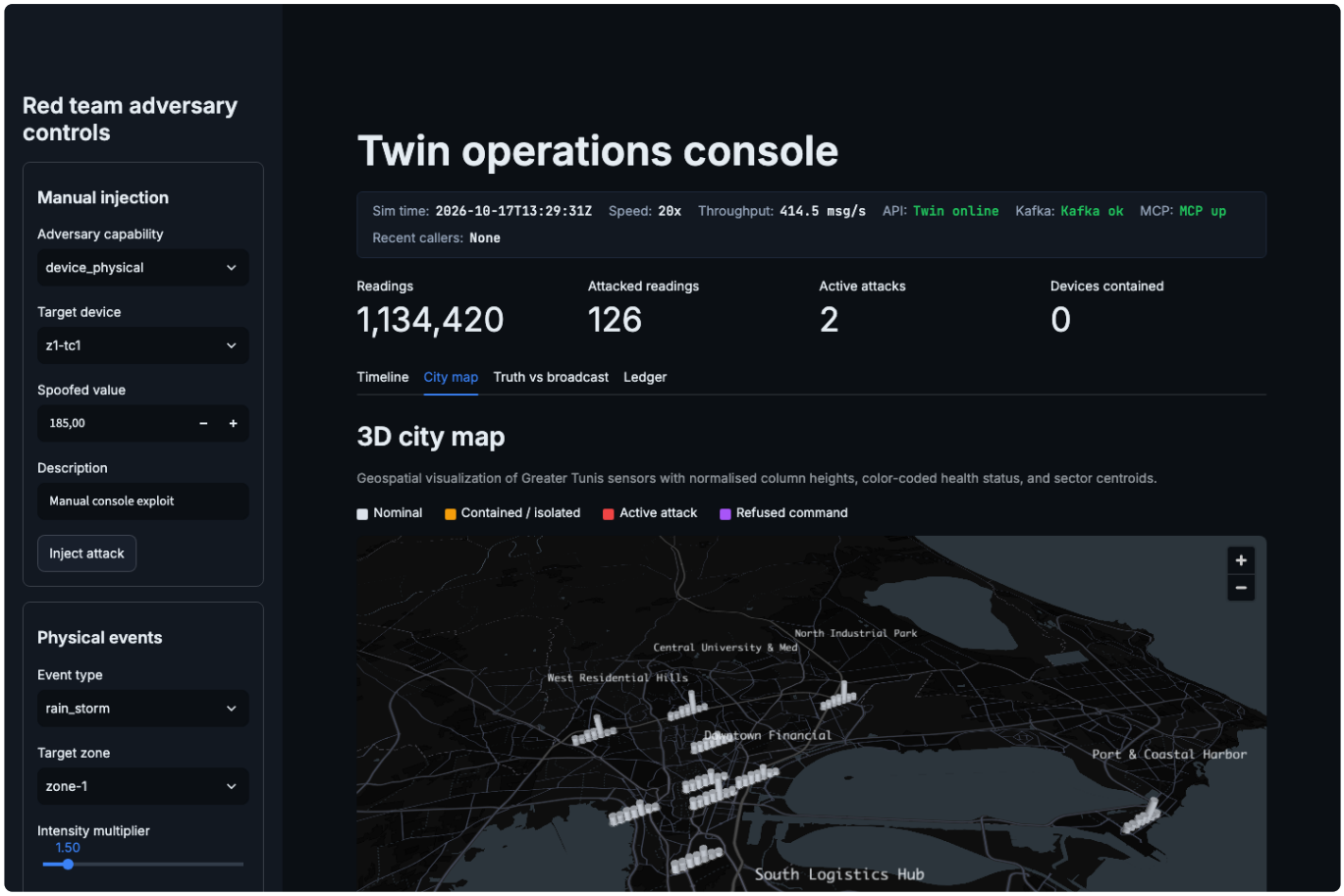
Above the timeline table, an **Incident chains** narrative list displays correlated chains sorted newest first. Each entry traces an incident lifecycle through arrows (`->`), showing detection, response latency, and escalation or resolution:
- `chain-1: z1-pl1 device_key spoofed to 180 -> Guardian quarantine_device +2 s (+1 more) -> no Brain action`
- `chain-3: grid_overload in zone-3 -> no Guardian action -> no Brain action -> escalated to blackout`
- `chain-2: rain_storm in zone-2 -> no Guardian action -> no Brain action -> resolved by cleared`

The main data grid presents columns for Time, Lane, Event, Target, Reaction, Summary, and Chain:
- Time: Simulation timestamp formatted as `HH:MM:SS`.
- Lane: Originating lane (`Twin` , `Guardian` , `Brain` , `Actuator`).
- Event: Event type identifier (`attack_inject` , `isolate_device` , `dispatch` , `set_valve` , etc.).
- Target: Device identifier or zone.
- Reaction: Reaction latency relative to incident origin (for example, `+2.0s`).
- Summary: Narrative description of the action and arguments.
- Chain: Correlated incident chain identifier (`chain-1` , `chain-2` , etc.).

Interactive row selection

Selecting any row in the operational data grid triggers an interactive inspection view below the table (**Inspect selected event payload**), displaying the complete underlying raw JSON record for deep forensic analysis.

Tab 2: City map

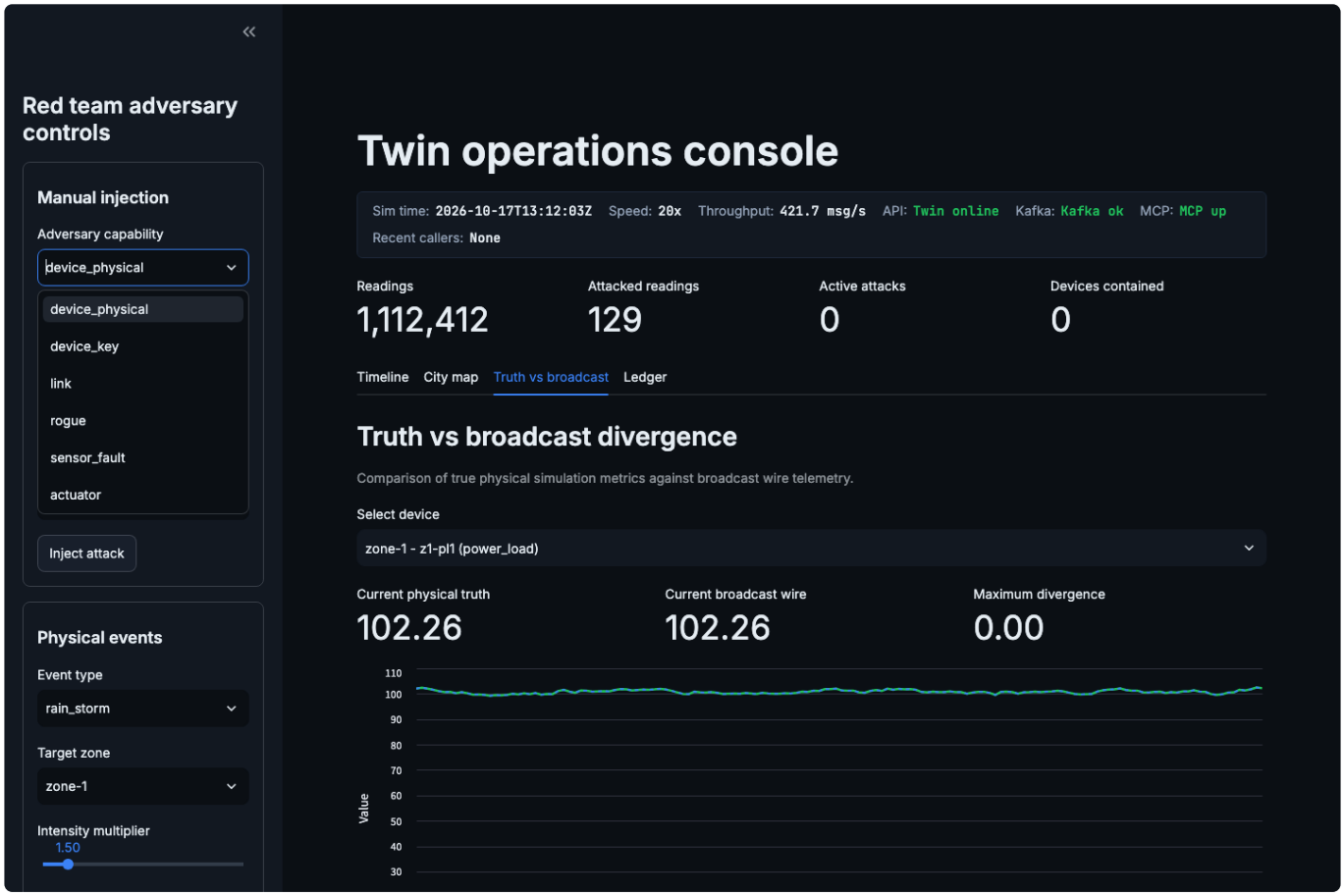


Greater Tunis 3D tactical map

The city map tab presents a 3D tactical visualization of Greater Tunis divided into six municipal zones:
- Downtown Financial (`zone-1`)
- Port and Coastal Harbor (`zone-2`)
- North Industrial Park (`zone-3`)
- West Residential Hills (`zone-4`)
- Central University and Medical District (`zone-5`)
- South Logistics Hub (`zone-6`)

Each sensor is represented as a 3D column positioned at its registered geographic coordinates. Column heights are individually normalised to each device type's configured range (`[min_val, max_val]`), preventing disparate units (such as megawatts and air quality index) from distorting relative visual scale. Pillars reflect device status through semantic coloring (green for nominal, red for active attack, amber for contained or isolated, and purple for refused actuation). Interactive tooltips provide exact device identifiers, device types, broadcast values, physical units, and operational status.

Tab 3: Truth vs broadcast

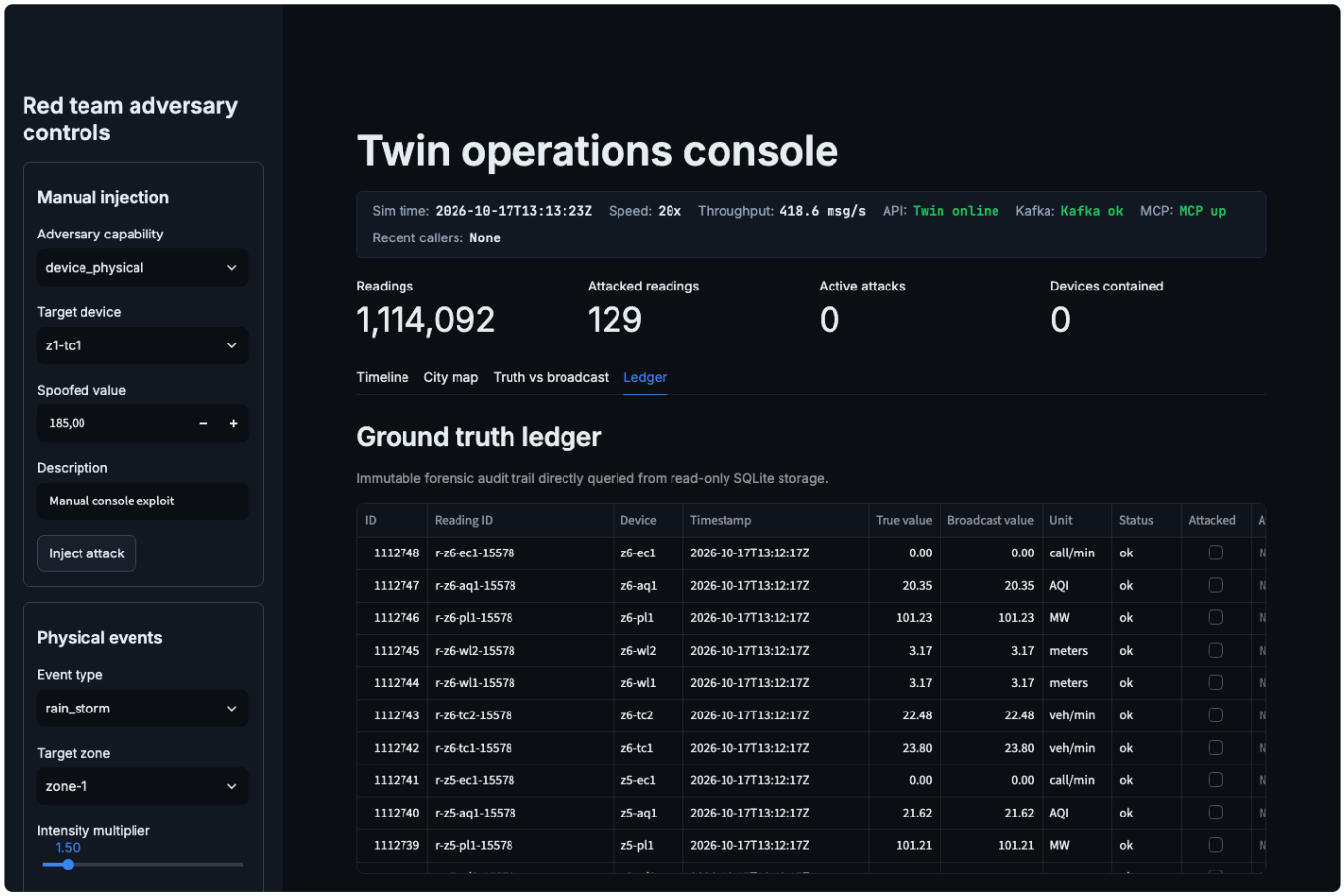


Ground truth versus broadcast divergence curves

The truth versus broadcast tab compares physical reality against wire transmissions:
- Solid green line: Ground truth value recorded in the private simulation ledger.
- Dashed blue line: Wire telemetry broadcast over Kafka.
- Red shaded region: Divergence between truth and wire, illustrating the impact of physical spoofing, drift, or replay attacks.

A device picker grouped by municipal zone allows rapid inspection of any of the 42 city sensors. Discrepancy metrics summarise the maximum observed deviation across the selected window.

Tab 4: Ledger



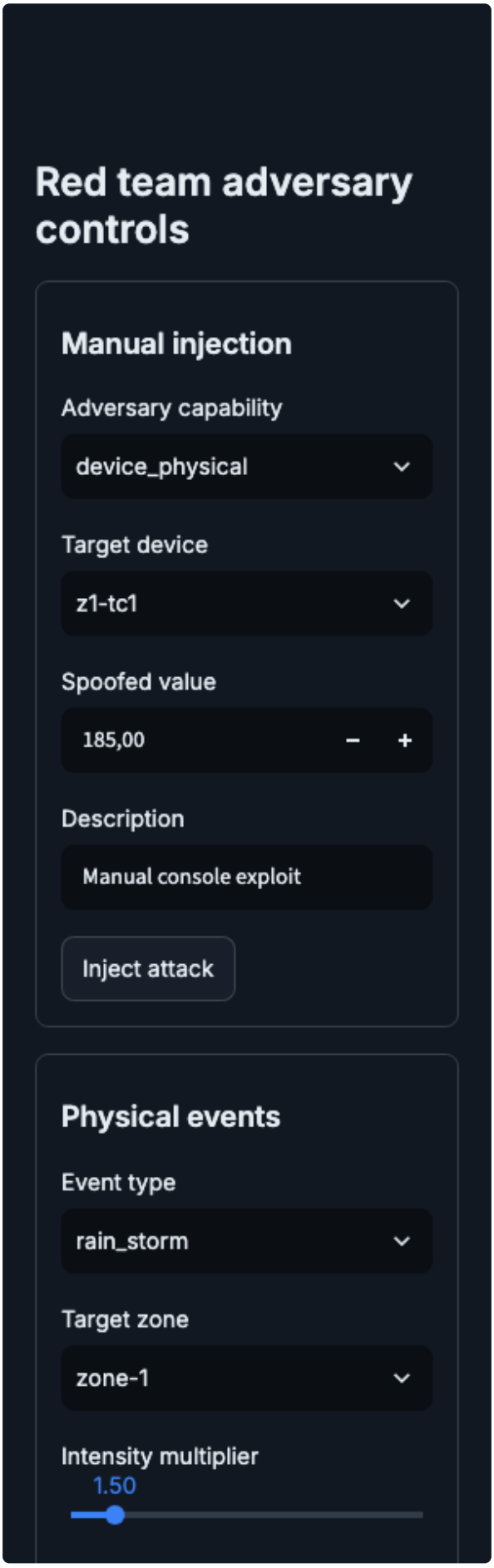
Read-only ground truth ledger audit trail

The ledger tab exposes the read-only forensic audit trail directly from `readings_truth`. Every record includes:
- Unique reading identifier (`r-{device_id}-{seq}`)
- Simulation timestamp
- Device identifier and municipal zone
- Ground truth value and broadcast wire value
- Attack classification flag and adversary capability type

Records with active attacks are highlighted with red row tinting, allowing operators and auditors to inspect historical incidents without risk of ledger corruption.

---

3. Red team adversary controls



Red team adversary controls

The sidebar provides authenticated controls to inject attacks and physical events into the simulation:

Manual injection

Enables targeted adversary capabilities against specific devices or zones:
- `device_physical` : Spoofs sensor output on wire while preserving truth in the ledger.
- `device_key` : Emulates cryptographic key theft and unauthorized signing.
- `link` : Manipulates network communication (corrupt bytes, invalid signature, replay, delay, frame drop).
- `rogue` : Introduces unauthorized transmitters into a target zone.
- `sensor_fault` : Injects calibration drift or stuck values.
- `actuator` : Forces actuation mechanisms to refuse supervisory commands while reporting success.

Physical events

Injects municipal conditions into selected zones:
- Types: `rain_storm` , `traffic_accident` , `fire` , `grid_overload` , `air_hazard` .
- Parameters: Target zone, intensity multiplier, and simulation duration.

Active attacks and events

Displays a real-time list of all currently active exploits and environmental conditions. Each active entry displays a dedicated **Clear** button for immediate individual deactivation. A global **Clear all threats** button flushes all active attacks and events simultaneously, immediately releasing any queued delay frames and restoring nominal sensor kinetics.

---

4. Console environment variables

The operations console accepts the following configuration environment variables:

Variable	Default	Description
`TWIN_API_URL`	`http://localhost:8100`	Red Team REST API endpoint used for querying journal entries, active threats, and status.
`TWIN_MCP_URL` / `MCP_URL`	`http://localhost:8000`	Model Context Protocol server endpoint used for device queries and command execution.
`GROUND_TRUTH_DB_PATH`	`ground_truth.db`	Filesystem path to the local SQLite simulation ledger.
`RED_TEAM_TOKEN`	*(unset)*	Literal secret token for authenticated Red Team REST API endpoints.
`RED_TEAM_TOKEN_FILE`	*(unset)*	File path containing the secret token (in Docker Compose, written to `/app/data/red_team.token`).
`TIMELINE_LOG_DIR`	*(unset)*	Optional directory path to load external partner log entries for display in the timeline.

---

5. Running the demo and caller attribution

Token configuration for clean timeline lanes

To route automated test harness actions into the dedicated **Test harness** lane rather than polluting the operational lanes, configure distinct principal tokens:

```
export MCP_AUTH_TOKENS="brain:tok-b,guardian:tok-g,scenario:tok-s"
export RED_TEAM_TOKEN="$(docker exec twin_layer cat /app/data/red_team.token)"
```

When running scenarios or tests, authenticate the test runner with `scenario:tok-s`. Tool calls issued by the test runner (such as initial settle steps, baseline resets, or test-time device reinstatements) are automatically attributed to caller `scenario` and categorized under the collapsed **Test harness** lane, while partner actions from Layer 2 and Layer 3 appear in the **Brain** and **Guardian** lanes.

Running with live mocks for manual testing

For manual testing and demonstration, you can run standalone mock processes:

```
# Start the live Guardian mock in one terminal:
python3 -m twin.mocks.mock_guardian

# Start the live City Brain mock in another terminal:
python3 -m twin.mocks.mock_brain
```

While these mocks are running, you can inject attacks or physical events through the console sidebar and watch the mocks react in real time on the Timeline and Map tabs.

Important: Stop standalone mocks before running automated scenarios. If you run automated test scripts with `--with-mocks`, stop any standalone mock processes first (`pkill -f "twin.mocks.mock_"`). Otherwise, multiple mock processes will consume from the same Kafka consumer groups or issue conflicting actuation commands.

Containment persistence across restarts

Device containment enforced by Guardian (or the console) is fully persistent. When a device is isolated or quarantined, its status is updated in the SQLite ledger (`devices.status`) and recorded in `containment_log`. If the Twin container or simulator process restarts on an existing ledger, all device containment states and rogue hardware blocklists are restored before the first simulation tick. Devices do not revert to nominal status until an authorized caller explicitly invokes `reinstate_device` or `release_device`.

---

6. Operational verification workflow

To verify complete end-to-end telemetry and defense responses:
1. Start the Twin unified runtime and console.
2. In the sidebar, select target device `z1-tc1`, capability `device_physical`, and enter a spoofed value (such as 185 veh/min). Click "Inject attack".
3. Observe the Timeline tab: A new incident chain appears showing the attack injection, the attacked reading, and the corresponding Guardian verdict.
4. Switch to the City map tab: The column for `z1-tc1` turns red.
5. Switch to the Truth vs broadcast tab: View the divergence curve showing real physical traffic (~45 veh/min) against the spoofed wire reading (185 veh/min).
6. Return to the sidebar and click "Clear all threats": The system returns to nominal baseline across all tabs and metrics.